HTTP API

The server speaks one wire format for songs: `.sng`, whatever shape a song was found in on disk. That is the design commitment in ADR-001 — an unmodified YARG reads a `.sng` natively, so the sync client of Phase 2b can write ordinary files into an ordinary songs folder and the game needs no change at all.

Everything below is v1 and unversioned only in the sense that `/api/v1` is the version.

Endpoints

Method	Path	What it is for
`GET`	`/healthz`	Liveness. Returns `ok`.
`GET`	`/version`	The build's version string.
`GET`	`/api/v1/library`	What was indexed, and what could not be.
`GET`	`/api/v1/songs`	Browse and search.
`GET`	`/api/v1/songs/{chart_hash}`	Every package sharing a chart hash.
`POST`	`/api/v1/have`	Bulk "what am I missing".
`GET`	`/song/{chart_hash}.sng`	The bytes.

`GET /api/v1/library`

```
{
  "songs": 22,
  "distinct_charts": 22,
  "duplicate_packages": 0,
  "built_at": "2026-09-05T20:11:05Z",
  "problems": [],
  "sort_attributes": ["name", "artist", "album", "artist_album", "genre", "subgenre",
                      "year", "charter", "playlist", "source", "song_length", "date_added"]
}
```

`songs` counts packages; `distinct_charts` counts songs as YARG identifies them, and the two differ whenever two packages share a chart.

`problems` **is the important field.** It names every directory or archive the scan could not read. A library that quietly indexes 9,000 of 10,000 songs is indistinguishable from one that has 9,000 songs, and an operator has no way to find out which — so failures are surfaced here rather than logged once at start and forgotten.

`GET /api/v1/songs`

Parameter	Default	Meaning
`q`	—	Free text. Matched against name, artist, album, genre, subgenre, charter, source and playlist.
`sort`	`name`	One of the twelve attributes above. Anything else is a **400**.
`order`	`asc`	`asc` or `desc`. Anything else is a 400.
`limit`	50	Capped at 500.
`offset`	0	Past the end is an empty page, not an error.

```
{ "total": 22, "offset": 0, "limit": 50, "sort": "artist", "order": "asc",
  "query": "", "songs": [ /* catalog entries */ ] }
```

An unrecognised `sort` is refused rather than silently replaced with the default. A client that asked for one order and was handed another without being told has no way to notice.

On ordering. Results come back in the order the *client* would show them: values are normalised exactly as YARG.Core's `SortString` does — leading article dropped, diacritics folded, rich-text markup stripped — and the tie-breakers are upstream's own comparer chains. See `internal/sortkey` and ADR-002 for what is reproduced and for the two places this server deliberately differs.

On matching. Only the *folding* is the client's, so `q=bjork` finds "Björk" and `q=the beatles` finds "The Beatles". The matching itself — a substring test across those eight attributes — is this server's own. Upstream's search logic has not been read and no parity is claimed for it. A query does not match across a field boundary: `q=blur blur` finds nothing even when the artist and the album are both "Blur".

`GET /api/v1/songs/{chart_hash}`

Returns a **list**, because song identity is deliberately many-to-one:

```
{ "chart_hash": "0856db78…", "count": 2, "songs": [ /* … */ ] }
```

Two packages with the same chart and different audio are the same song to YARG — its own cache is `hash -> List<SongEntry>`. Returning only the first would hide the other from every client that asked.

`POST /api/v1/have`

The client sends the chart hashes it already holds; the answer is everything in the library that was not in that list. One round trip, whatever the size of either side.

```
{ "chart_hashes": ["0856db78…", "b434fc60…"] }
```

```
{ "library_total": 22, "missing": ["2db04926…", "…"], "missing_count": 20 }
```

Hashes are compared case-insensitively and surrounding whitespace is ignored, because a client assembling a list from its own cache should not be punished for either. `missing` is sorted, so the same question twice gives the same bytes.

Chart hash, not package hash, on purpose: a client that has the chart can play the song, and re-sending it a package that differs only in album art is bandwidth spent on nothing.

An unknown field in the body is a **400** rather than being ignored — a client that sent `chart_hash` for `chart_hashes` would otherwise be told, plausibly and wrongly, that it is missing the entire library.

`GET /song/{chart_hash}.sng`

The package bytes, always as `.sng`. A song stored as a loose folder is packed on demand and the archive is cached; packing copies the chart byte for byte, so the song's identity is unchanged by it.

- `ETag` is the package hash — a hash of the content, so it is the same on every server holding the same package and survives a rescan, a restart and a cache wipe.
- `Range` is supported, so an interrupted download resumes rather than starting again.
- **300 Multiple Choices** when the chart hash is shared by several packages. The response lists them; `?package=<package_hash>` names one. The server does not choose, because choosing would hand different clients different audio for the same request.
- **404** when the hash is unknown, and also when the index points at a file that has since moved — with a message saying to rescan, because that is the fix.

What is not here yet

Ingest (`POST` of a folder, `.sng` or `.zip`), a config file, and authentication. See `docs/ROADMAP.md`. The server is currently read-only and unauthenticated: run it on a LAN, not on the internet.